Ex.No 1: The built-in R data set quakes gives the locations of earthquakes off of Fiji in the 1960‘s. Create a plot of the locations of these earthquakes, showing depth with color and magnitude with size.

Aim:

To create a plot of the earthquake locations from the quakes dataset in R, displaying depth with color and magnitude with size.

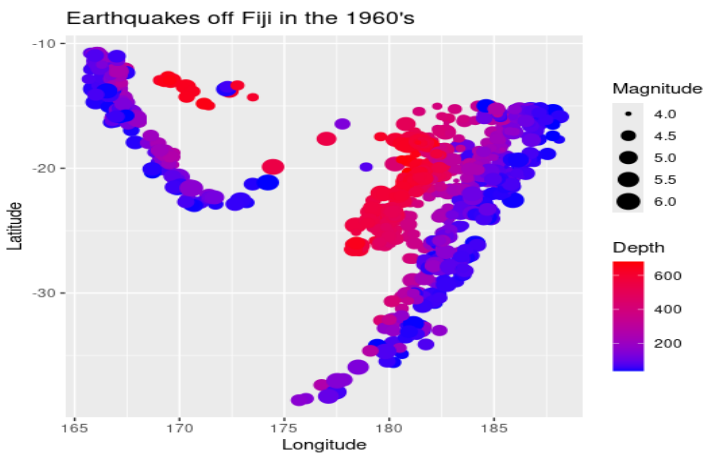
Algorithm

1. Load the quakes dataset.
2. Define the color and size scales based on depth and magnitude, respectively.
3. Plot the earthquake locations using longitude and latitude.
4. Assign depth to color and magnitude to size.
5. Add labels and title to the plot.

Code

```
#install package
install.packages("ggplot2")

# Load required library
library(ggplot2)

# Load the quakes dataset
data(quakes)

# Plot earthquakes with depth as color and magnitude as size
ggplot(quakes, aes(x = long, y = lat, color = depth, size = mag)) +
  geom_point() +
  scale_color_gradient(low = "blue", high = "red") +
  labs(title = "Earthquakes off Fiji in the 1960's",
       x = "Longitude", y = "Latitude", color = "Depth", size = "Magnitude")
```

Output:



Result:

Thus a plot of the earthquake locations from the quakes dataset in R, displaying depth with color and magnitude with size has been created and executed successfully.

Ex.No 2: Create a boxplot of highway mileage for each different cylinder in mtcars, and display on one plot with highway mileage on the y-axis and cylinder on the x-axis

Aim

To create a boxplot of highway mileage for each different cylinder in the mtcars dataset and display it on one plot with highway mileage on the y-axis and cylinder on the x-axis.

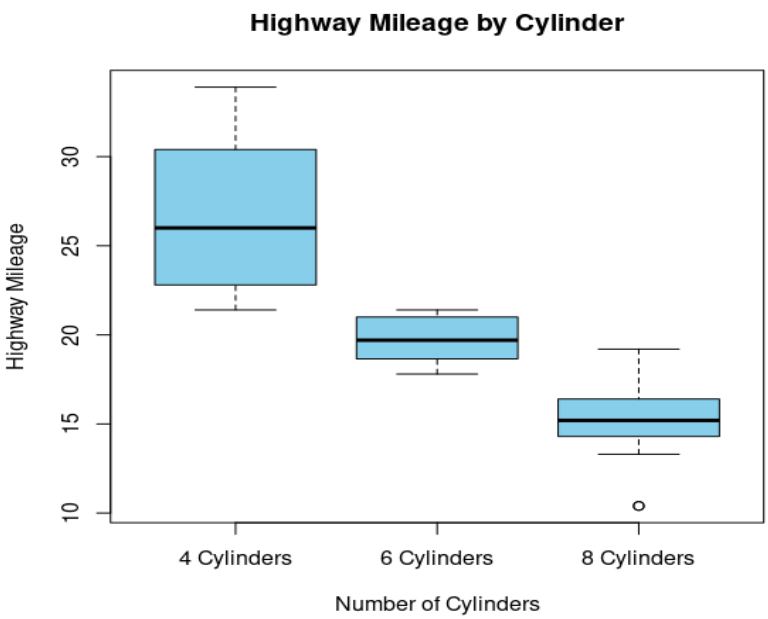
Algorithm:

1. Split the data by the cylinder variable.
2. Calculate the highway mileage for each cylinder group.
3. Create a boxplot with highway mileage on the y-axis and cylinder on the x-axis.

Code

```
# Step 1: Split the data by cylinder variable
cylinder_groups <- split(mtcars$mpg, mtcars$cyl)

# Step 2: Create a boxplot of highway mileage for each cylinder group
boxplot(cylinder_groups,
        main = "Highway Mileage by Cylinder",
        xlab = "Number of Cylinders",
        ylab = "Highway Mileage",
        names = c("4 Cylinders", "6 Cylinders", "8 Cylinders"),
        col = "skyblue",
        border = "black")
```

Output



Result:

Thus a boxplot of highway mileage for each different cylinder in mtcars, and display on one plot with highway mileage on the y-axis and cylinder on the x-axis has been created and executed successfully.

Ex.No 3: Create a barplot of the word lengths of the words in the data set, faceted by novel using austen data set from the fosdata package

Aim:

To create a barplot of the word lengths of the words in the data set, faceted by novel using austen data set from the fosdata package

Algorithm:

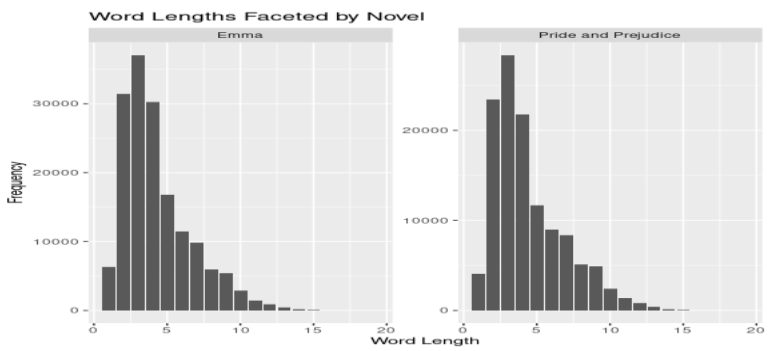
Install and load the "fosdata" package.

1. Load the "austen" dataset.
2. Calculate the word lengths.
3. Create a barplot faceted by novel.

Code

```
# Step 1: Install and load the "fosdata" package

install.packages("remotes")
remotes::install_github("speegled/fosdata")
library(fosdata)

# Step 2: Load the "austen" dataset
data(austen)

# Step 3: Calculate the word lengths
austen$word_length <- nchar(austen$word)

# Step 4: Create a barplot faceted by novel
library(ggplot2)
ggplot(austen, aes(x = word_length)) +
  geom_bar() +
  facet_wrap(~ novel, scales = "free") +
  labs(title = "Word Lengths Faceted by Novel",
       x = "Word Length",
       y = "Frequency")
```

Output:



Result:

Thus a barplot of the word lengths of the words in the data set, faceted by novel using austen data set from the fosdata package has been created and executed successfully.

Ex.No 4: The pres_election data set gives voting results from the 2010-2016 U.S. presidential elections. Produce five bar charts, one for each election, that show the total number of votes received by each political party. Use facet_wrap to put all five charts into the same visualization.

Aim:

To create five bar charts, one for each election year, showing the total number of votes received by each political party using the "pres_election" dataset, and to put all five charts into the same visualization using facet_wrap

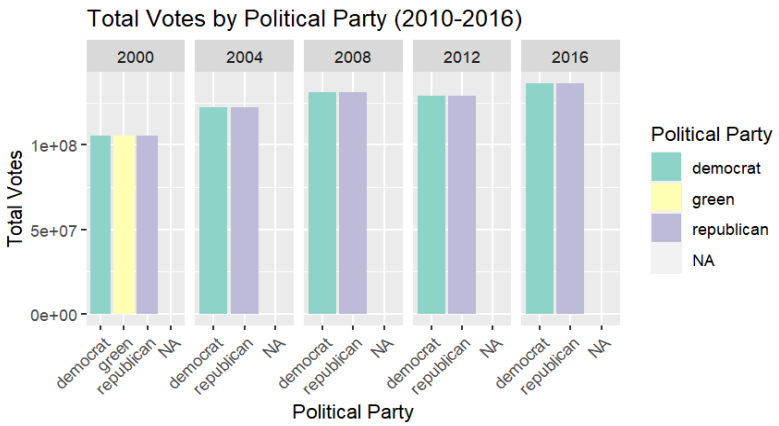
Algorithm:

1. Load the "pres_election" dataset.
2. Aggregate the total number of votes received by each political party for each election year.
3. Create a bar chart for each election year.
4. Use facet_wrap to put all five charts into the same visualization.

Code:

```
library(fosdata)
# Load the pres_election dataset
data(pres_election)

# Load the required library
library(dplyr)
library(ggplot2)

# Aggregate the total number of votes received by each political party for each election year
party_votes <- pres_election %>%
  group_by(year, party) %>%
  summarise(total_votes = sum(totalvotes))

# Create bar charts for each election year
ggplot(party_votes, aes(x = party, y = total_votes, fill = party)) + geom_bar(stat = "identity") +
facet_wrap(~ year, scales = "free_x", nrow = 1) + labs(title = "Total Votes by Political Party (2010-
2016)", x = "Political Party", y = "Total Votes", fill = "Political Party") + theme(axis.text.x =
element_text(angle = 45, hjust = 1)) +   scale_fill_brewer(palette = "Set3")
```

Output:



Result:

Thus the five bar charts, one for each election year, showing the total number of votes received by each political party using the "pres_election" dataset, and to put all five charts into the same visualization using facet_wrap has been created and executed successfully.

Ex.No 5: Create a scatterplot of highway mileage versus city mileage colored by the number of cylinders, using the mtcars data set. Experiment using categorical and sequential coloring.

Aim:

To create a scatterplot of highway mileage versus city mileage colored by the number of cylinders using the mtcars dataset using categorical and sequential coloring.

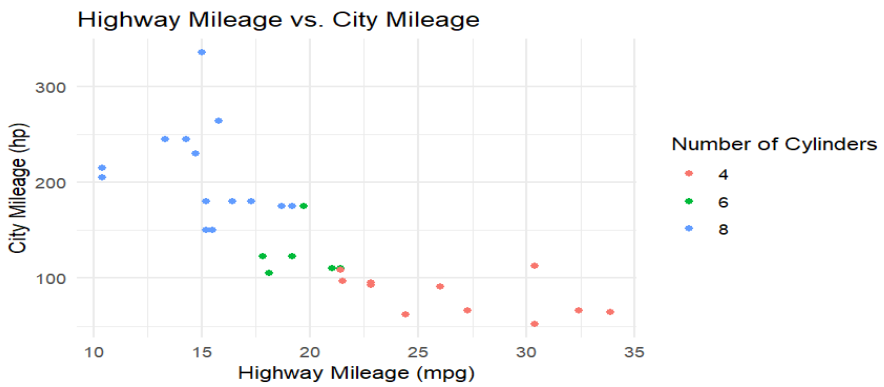
Algorithm:

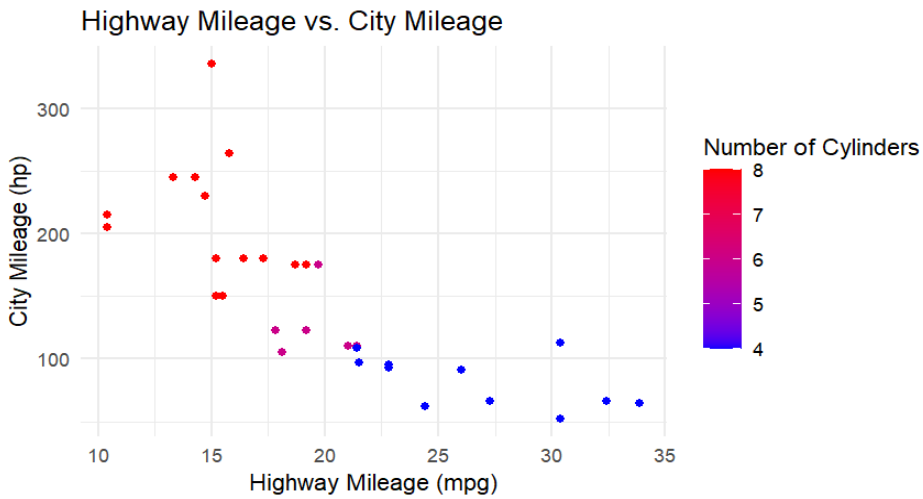
1. Load the mtcars dataset.
2. Prepare the data by creating a dataframe with the required variables (highway mileage, city mileage, and number of cylinders).
3. Create a scatterplot with highway mileage on the x-axis, city mileage on the y-axis, and color-coded by the number of cylinders.
4. Plot the graph with categorical and sequential coloring.

Code:

```
install.packages("ggplot2")
library(ggplot2)
# Load the mtcars dataset
data(mtcars)

# Scatterplot with categorical coloring by number of cylinders
ggplot(mtcars, aes(x = mpg, y = hp, color = factor(cyl))) +
  geom_point() +
  labs(title = "Highway Mileage vs. City Mileage",
       x = "Highway Mileage (mpg)",
       y = "City Mileage (hp)",
       color = "Number of Cylinders") +
  theme_minimal()

# Scatterplot with sequential coloring by number of cylinders
ggplot(mtcars, aes(x = mpg, y = hp, color = cyl)) +
  geom_point() +
  labs(title = "Highway Mileage vs. City Mileage",
       x = "Highway Mileage (mpg)",
       y = "City Mileage (hp)",   color = "Number of Cylinders") +   scale_color_gradient(low =
"blue", high = "red") +    theme_minimal()
```

Output:





In the first scatterplot, each cylinder group is represented by a different color. In the second scatterplot, the number of cylinders is represented by a sequential color scale ranging from blue (for fewer cylinders) to red (for more cylinders).

Result:

Thus a scatterplot of highway mileage versus city mileage colored by the number of cylinders using the mtcars dataset using categorical and sequential coloring has been created and executed successfully.

Ex.No 6: In Emma, restrict to words that have non-zero sentiment score. Create a scatterplot of the percentage of words that have a positive sentiment score versus chapter. Add a line using geom_line or geom_smooth and explain your choice using austen data set from the fosdata package

Aim:

To create a scatterplot of the percentage of words in "Emma" by Jane Austen that have a positive sentiment score versus chapter, and then add a line representing the trend using geom_smooth.

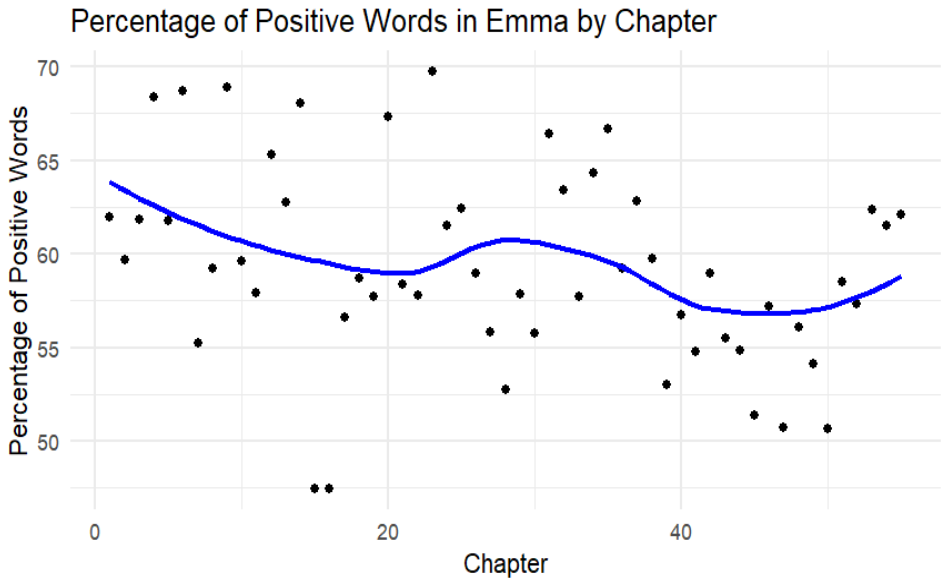
Algorithm:

1. Load the "fosdata" package and extract the "emma" dataset.
2. Filter the dataset to include only words with non-zero sentiment scores.
3. Calculate the percentage of words with positive sentiment scores for each chapter.
4. Create a scatterplot of the percentage of positive words versus chapter.
5. Add a line representing the trend using geom_smooth.

Code:

```
# Load the fosdata package
library(fosdata)

# Load the austen dataset
data(austen)

# Load required libraries
library(dplyr)
library(ggplot2)

# Filter the dataset to include only the rows corresponding to the novel "Emma"
emma_data <- austen %>%
  filter(novel == "Emma")

# Filter the dataset to include only words with non-zero sentiment scores
emma_filtered <- emma_data %>%
  filter(sentiment_score != 0)

# Calculate the percentage of words with positive sentiment scores for each chapter
positive_percentage <- emma_filtered %>%
  group_by(chapter) %>%
  summarise(positive_percentage = sum(sentiment_score > 0) / n() * 100)

# Create scatterplot of percentage of positive words versus chapter
ggplot(positive_percentage, aes(x = chapter, y = positive_percentage)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, color = "blue") +
  labs(title = "Percentage of Positive Words in Emma by Chapter",
       x = "Chapter",
       y = "Percentage of Positive Words") +
  theme_minimal()
```

Output:



Result:

Thus a scatterplot of the percentage of words in "Emma" by Jane Austen that have a positive sentiment score versus chapter, and then add a line representing the trend using geom_smooth has been created and executed successfully.

Ex.No 7: Make a scatterplot showing CO2 uptake as a function of concentration level for the built-in data set CO2. Include a smoothed fit line and color by Type. Facet your plot to one plot for each Plant.

Aim:

To make a scatterplot showing CO2 uptake as a function of concentration level for the built-in data set CO2 and included a smoothed fit line and color by Type.

Algorithm :

1. Load the necessary libraries: Load the required libraries, particularly ggplot2 for creating the plot.

2. Load the dataset: Load the CO2 dataset into your R environment.

3. Prepare the data: Ensure the 'Plant' column is treated as a factor for proper facetting.

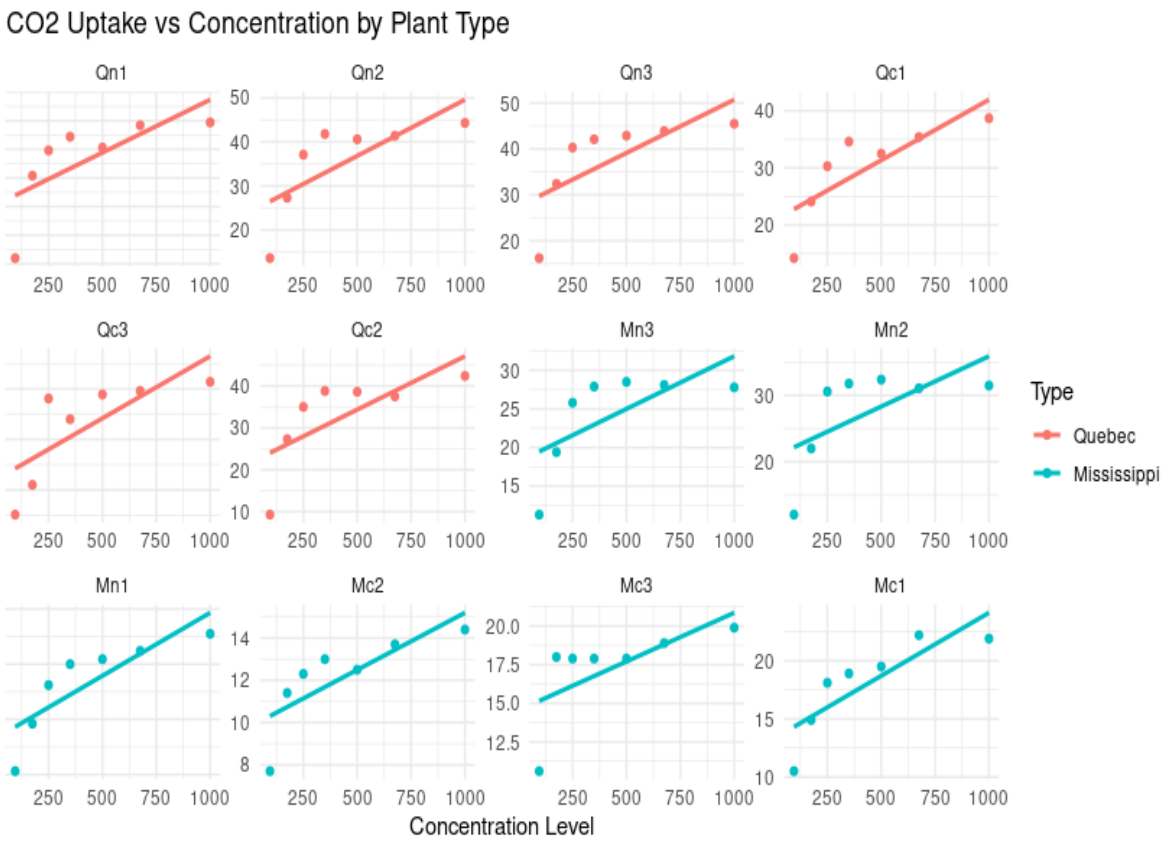
4. Create the plot: Use ggplot to create a scatterplot (geom_point) of CO2 uptake against concentration level, color-coded by Type. Add a smoothed fit line (geom_smooth) using linear regression. Facet the plot by Plant.

5. Add labels and theme: Add appropriate labels for the axes and a title for the plot. Optionally, customize the theme for better aesthetics.

Program :

```
# Load necessary libraries
library(ggplot2)
# Load CO2 dataset
data(CO2)
# Convert Plant column to factor for proper facetting
CO2$Plant <- as.factor(CO2$Plant)
# Create the plot
ggplot(CO2, aes(x = conc, y = uptake, color = Type)) +
geom_point() + # Scatterplot
geom_smooth(method = "lm", se = FALSE) + # Add a linear model fit line
facet_wrap(~Plant, scales = "free") + # Facet by Plant
labs(title = "CO2 Uptake vs Concentration by Plant Type",
x = "Concentration Level",
y = "CO2 Uptake") + # Labels
theme_minimal() # Theme
```

Output :



Result:

Thus a scatterplot showing CO2 uptake as a function of concentration level for the built-in data set CO2 and included a smoothed fit line and color by Type has been created and executed successfully.

Ex No :8 Consider the ecars data set create a visualization showing scatterplots with the charge TimeHrs variable on the x axis and the kwhTotal variable on the y axis. Facet your visualization with one plot per day of week and platform. Remove the web platform cars, so you have 14 facets in two rows and seven columns.

Aim:

To create visualization that shows scatterplots with the charge TimeHrs variable on the x axis and the kwhTotal variable on the y axis using ecars data set.

Algorithm:

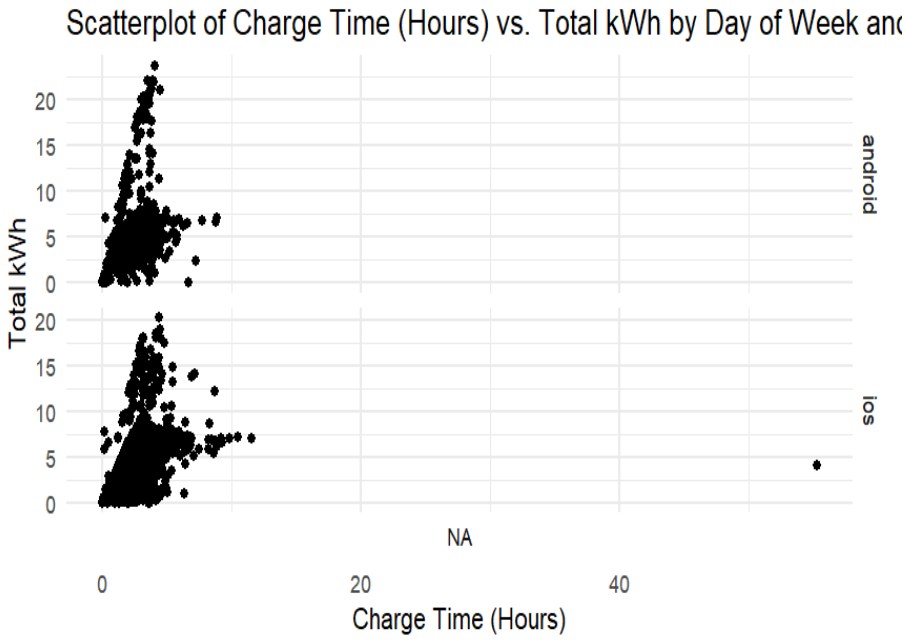
Create Scatterplot Visualization with mpg Dataset

1. Load Necessary Libraries Import the required libraries, particularly ggplot2 for data visualization.
2. Load the Dataset Load the mpg dataset into your R environment.
3. Prepare the Data a. Filter out the rows with the class 'suv' to remove them from the dataset. b. Ensure 'manufacturer' column is treated as a factor and reorder the levels as needed.
4. Create the Plot a. Use ggplot to create a scatterplot with 'hwy' (highway miles per gallon) on the x-axis and 'cty' (city miles per gallon) on the y-axis. b. Facet the visualization by 'manufacturer'.
5. Customize the Plot Add appropriate labels for the axes and a title for the plot. Optionally, customize the theme for better aesthetics.

Code

```
# Load required libraries
library(dplyr)
library(ggplot2)
library(lubridate)   # For handling dates

# Load the ecars dataset
data(ecars)

# Filter out web platform cars
ecars_filtered <- ecars %>%
  filter(platform != "web")

# Convert dayOfWeek to factor with proper order
ecars_filtered$weekday <- factor(ecars_filtered$weekday, levels = c("Sunday", "Monday",
"Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"))

# Create the scatterplot with facets
ggplot(ecars_filtered, aes(x = chargeTimeHrs, y = kwhTotal)) +
  geom_point() +
  facet_grid(platform ~ weekday, scales = "free", space = "free", switch = "x") +
  labs(title = "Scatterplot of Charge Time (Hours) vs. Total kWh by Day of Week and Platform",
       x = "Charge Time (Hours)",
       y = "Total kWh") +
  theme_minimal()
```

Output:



Result:

Thus the visualization that shows scatterplots with the charge TimeHrs variable on the x axis and the kwhTotal variable on the y axis using ecars data set has been created and executed successfully.

Ex No 9: Consider the scotland_births data set in the fosdata package. This data set contains the number of births in Scotland by age of the mother for each year from 1945-2019.

a. Create a line plot of births by year from 1945-2019 for each age group represented in the data.
b. b. Highlight and color ages 20 and 30, and provide meaningful labels and titles

Aim:

To write a R code for creating line plots of births with different age group of the mother by highlighting ages 20 and 30 with different colors in the scotland_births dataset.

Algorithm:

1. Data Preparation: The provided dataset scotland_births is converted into a tidy format using the pivot_longer function from the tidyverse package. This function reshapes the data from wide to long format, making it easier to work with in subsequent steps. The columns representing the years are transformed into a single column named "year", and the corresponding values representing the number of births are stored in another column named "births".

2. Data Filtering: Since the dataset contains data for multiple years, only the years from 1945 onwards are considered. This is achieved by filtering the dataset to include only the rows where the year is greater than or equal to 1945.

3. Plotting: The ggplot function from the ggplot2 package is used to create the line plot. The x-axis represents the years, the y-axis represents the number of births, and each age group is represented by a different color. The geom_line function adds lines connecting the data points for each age group.

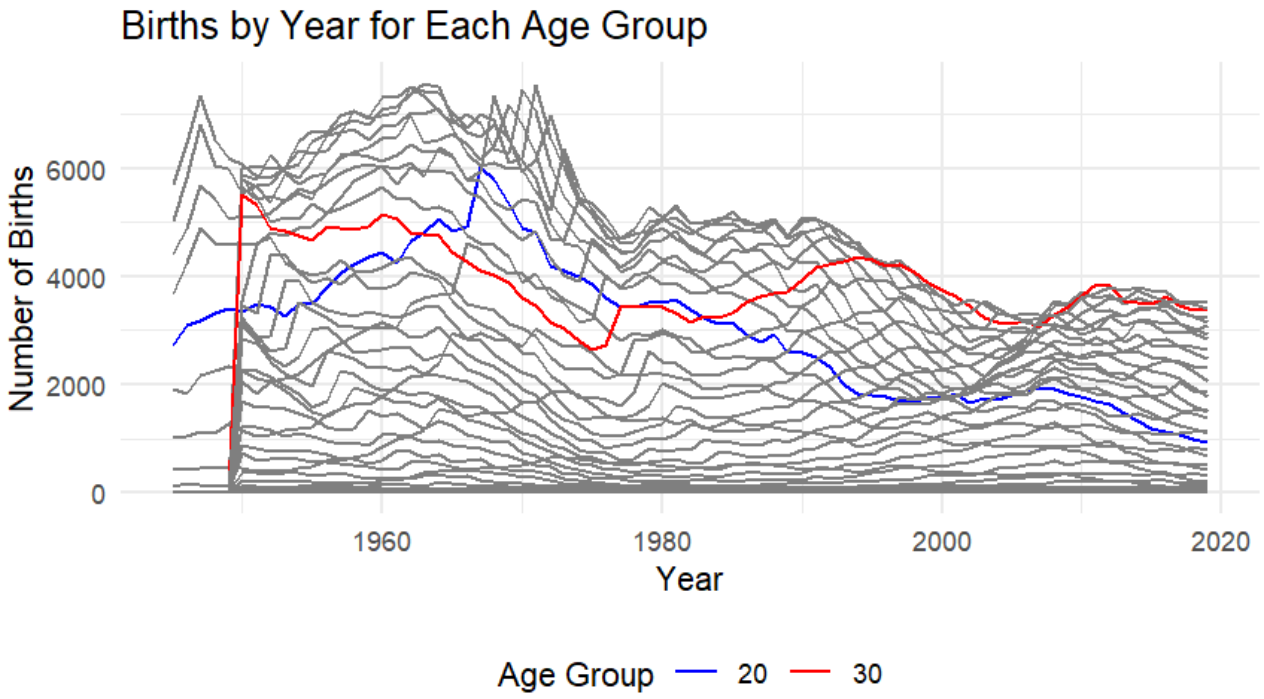
4. Customization: Additional customization options are applied to the plot, including manual color assignment to specific age groups (20 and 30-year-olds are highlighted in blue and red, respectively), minimal theme, and appropriate axis labels and title.

Code:

```
# Load necessary libraries
library(tidyverse)

# Convert the provided data into a tidy format
scotland_births_tidy <- pivot_longer(scotland_births,
                                     cols = -age,
                                     names_to = "year",
                                     values_to = "births") %>%
  mutate(year = as.numeric(str_remove(year, "x"))) %>%
  filter(year >= 1945)

# Plot births by year for each age group
ggplot(scotland_births_tidy, aes(x = year, y = births, color = as.factor(age))) +
  geom_line() +
  scale_color_manual(values = c("20" = "blue", "30" = "red", "other" = "gray")) +
  theme_minimal() +
  labs(title = "Births by Year for Each Age Group",
       x = "Year",
       y = "Number of Births",
```

```
        color = "Age Group") +
  theme(legend.position = "bottom")
```

Output:



Result:

Thus a code for creating line plots of births with different age group of the mother by highlighting ages 20 and 30 with different colors in the scotland_births dataset has been created and executed successfully.

Ex No 10: Consider the frogs data set in the fosdata package. This data was used to argue that a new species of frog had been found in a densely populated area of Bangladesh. Create a scatterplot of head length distance from tip of snout to back of mandible versus forearm length distance from corner of elbow to proximal end of outer palmar metacarpal tubercle, colored by species.

Aim:

To create a scatterplot of head length distance from tip of snout to back of mandible versus forearm length distance from corner of elbow to proximal end of outer palmar metacarpal tubercle, colored by species using the dataset (frogs) from the fosdata package in R.

Algorithm:

1. Load Required Libraries: Import the necessary libraries, such as ggplot2 and fosdata, to access the dataset and create the scatterplot.

2. Load Dataset: Load the frogs dataset from the fosdata package into R.

3. Create Scatterplot: Use the ggplot() function from the ggplot2 library to create the scatterplot. Specify the dataset (frogs) and map the variables hl (head length) to the x-axis and fal (forearm length) to the y-axis.

4. Color by Species: Use the color aesthetic inside the aes() function to color the points based on the species of frogs.

5. Customize Labels: Add appropriate labels to the x-axis, y-axis, and color legend to make the plot more informative.

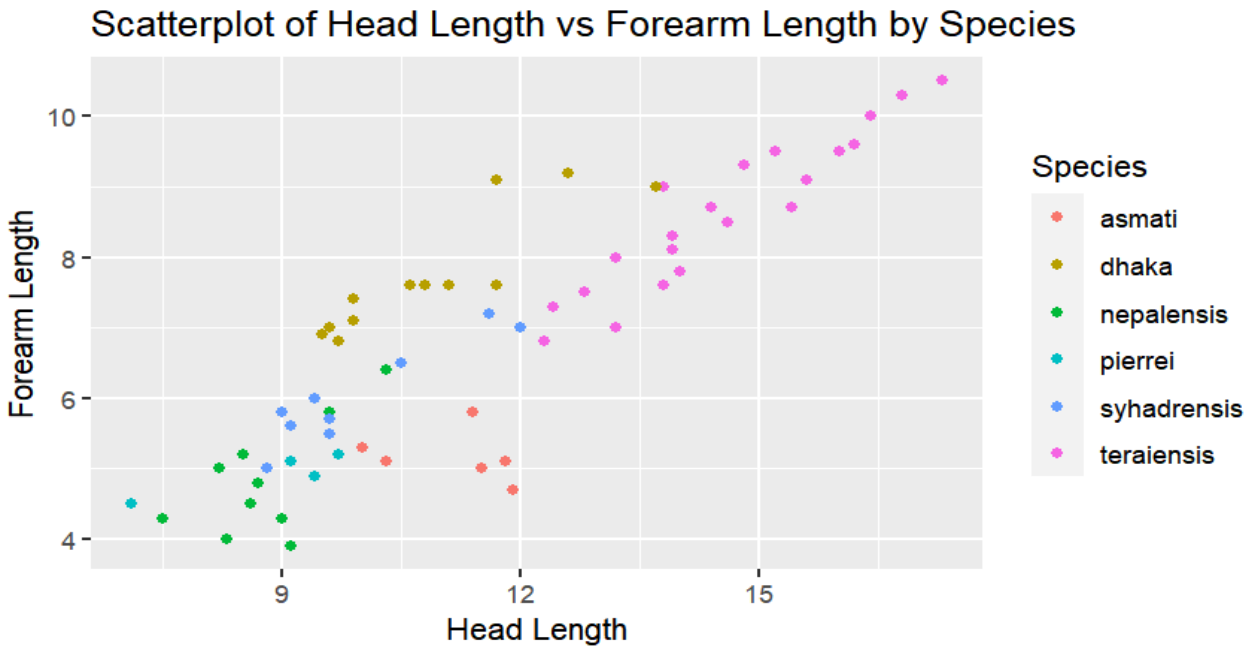
6. Add Title: Provide a title for the scatterplot to summarize the information being presented.

7. Display Plot: Use the appropriate function (e.g., print(), plot(), or show()) to display the scatterplot.

Code:

```
# Load required libraries
library(ggplot2)
library(fosdata)

# Create scatterplot
ggplot(data = frogs, aes(x = hl, y = fal, color = species)) +
  geom_point() +
  labs(x = "Head Length", y = "Forearm Length", color = "Species") +
  ggtitle("Scatterplot of Head Length vs Forearm Length by Species")
```

Output:



Result:

Thus a scatterplot of head length distance from tip of snout to back of mandible versus forearm length distance from corner of elbow to proximal end of outer palmar metacarpal tubercle, colored by species using the dataset (frogs) from the fosdata package in R.

Ex No 11: Use the babynames data set from the babynames package.

a. Make a line graph of the total number of babies of each sex versus year
b. Make a line graph of the number of different names used for each sex versus year
c. Make a line graph of the total number of babies with your name versus year. If your name doesn't appear in the data, use the name —Alexa
d. Make a line graph comparing the number of boys named Bryan and the number of boys named Brian from 1920 to the present.

Aim:

To create a line graph of various criteria from baby names dataset using R language.

Algorithm:

1) Install and load packages:
Check if the babynames package is installed.
If not, install and load the package.

2) Load required libraries:
Load the necessary libraries (dplyr and ggplot2).

3) Total number of babies by sex and year:
a.Use the babynames dataset.
b.Group by year and sex.
c.Summarize the total number of babies (total_babies) for each group.
d.Create a line graph using ggplot2 with years on the x-axis, total babies on the y-axis, and different lines for each sex.

4) Number of different names by sex and year:
a.Use the babynames dataset.
b.Group by year and sex.
c.Summarize the number of unique names (unique_names) for each group.
d.Create a line graph using ggplot2 with years on the x-axis, unique names on the y-axis, and different lines for each sex.

5) Total number of babies with a specific name (replace with your name or "Alexa"):
a.Specify the name to be analyzed (replace "YourName" with your actual name).
b.Check if the specified name exists in the dataset.
c.If not, set the name to "Alexa."
d.Filter the dataset for the specified name.
e.Group by year.
f.Summarize the total number of babies (total_babies) for each year.
g.Create a line graph using ggplot2 with years on the x-axis and total babies on the y-axis.

6) Comparison of boys named Bryan and Brian from 1920 to the present:
a.Use the babynames dataset.
b.Filter for male names ("M").
c.Filter for names "Bryan" and "Brian."
d.Filter for years greater than or equal to 1920.
e.Group by year and name.
f.Summarize the total number of babies (total_babies) for each group.
g.Create a line graph using ggplot2 with years on the x-axis, total

babies on the y-axis, and different lines for "Bryan" and "Brian."

Code

a. Make a line graph of the total number of babies of each sex versus year

```
# Install and load the babynames package if not already installed
if (!require(babynames)) {
  install.packages("babynames")
  library(babynames)
}
# Load required libraries
library(dplyr)
library(ggplot2)

# a) Make a line graph of the total number of babies of each sex versus year
total_babies <- babynames %>%
  group_by(year, sex) %>%
  summarize(total_babies = sum(n))

ggplot(total_babies, aes(x = year, y = total_babies, color = sex, group = sex)) +
  geom_line() +
  labs(title = "Total Number of Babies by Sex and Year",
       x = "Year",
       y = "Total Babies")
```

b. Make a line graph of the number of different names used for each sex versus year

```
# Load required libraries
library(dplyr)
library(ggplot2)

# Group the data by year and sex, count the number of unique names
unique_names <- babynames %>%
  group_by(year, sex) %>%
  summarize(num_unique_names = n_distinct(name))

# Plot the line graph
ggplot(unique_names, aes(x = year, y = num_unique_names, color = sex, group = sex)) +
  geom_line() +
  labs(title = "Number of Different Names Used by Sex and Year",
       x = "Year",
       y = "Number of Unique Names")
```

c. Make a line graph of the total number of babies with your name versus year. If your name doesn‘t appear in the data, use the name —Alexa

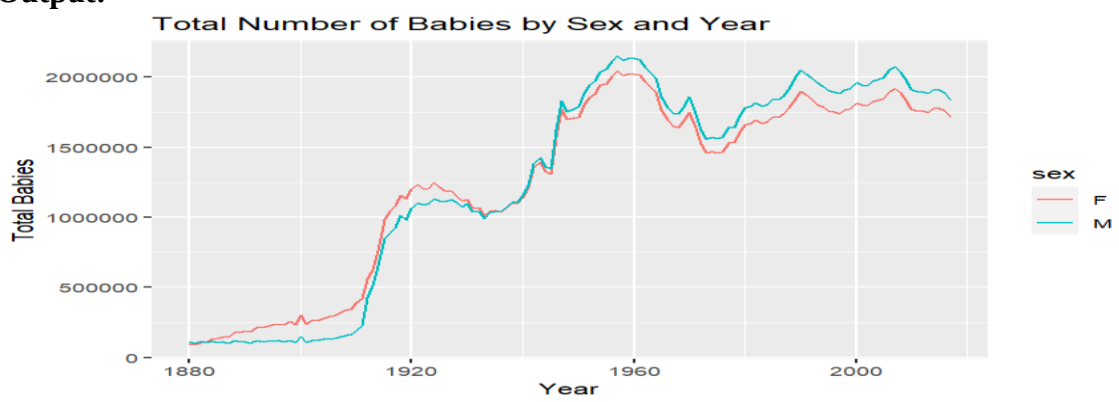
```
# Load required libraries
library(dplyr)
library(ggplot2)

# Set your name
your_name <- "Your Name"
```

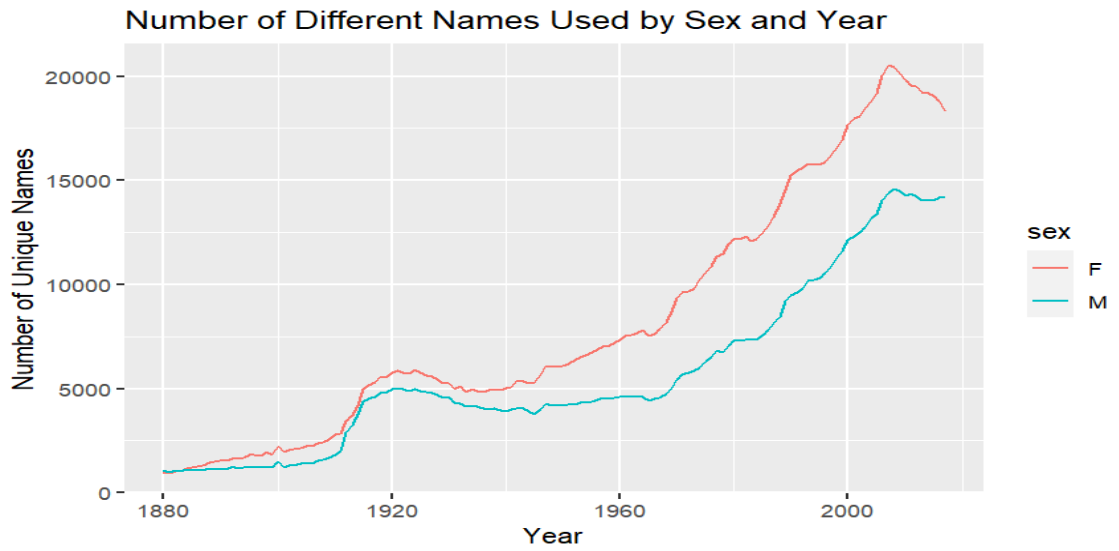
```
# Filter the dataset for your name, or "Alexa" if your name doesn't appear
filtered_data <- babynames %>%
  filter(name == ifelse(your_name %in% unique(babynames$name), your_name, "Alexa"))

# Group the filtered data by year and calculate the total number of babies
total_babies <- filtered_data %>%
  group_by(year) %>%
  summarize(total_babies = sum(n))

# Plot the line graph
ggplot(total_babies, aes(x = year, y = total_babies)) +
  geom_line() +
  labs(title = paste("Total Number of Babies Named", ifelse(your_name %in% unique(babynames$name), your_name, "Alexa"), "by Year"),
       x = "Year",
       y = "Total Babies")
```

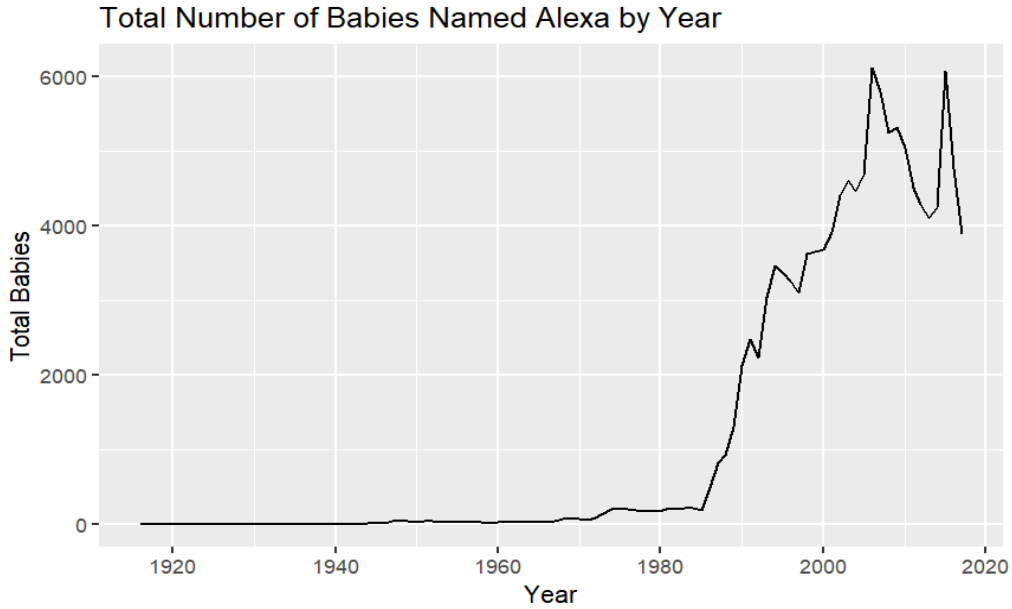
d. Make a line graph comparing the number of boys named Bryan and the number of boys named Brian from 1920 to the present

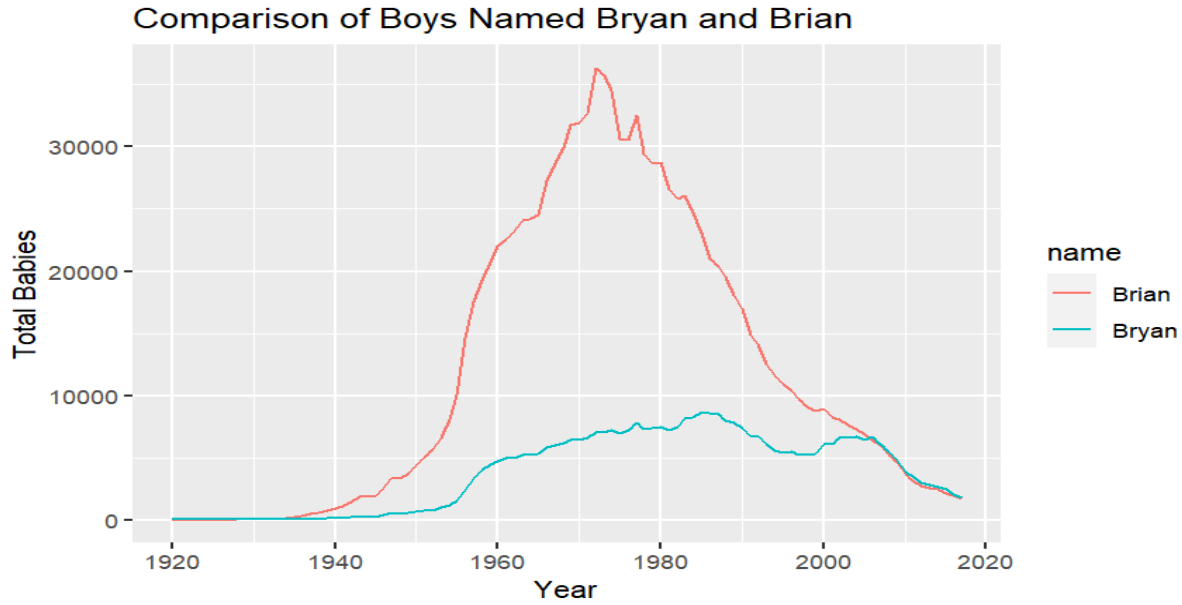
```
# Load required libraries
library(dplyr)
library(ggplot2)

# Filter the dataset for names "Bryan" and "Brian" for boys from 1920 to the present
filtered_data <- babynames %>%
  filter(name %in% c("Bryan", "Brian"), sex == "M", year >= 1920)

# Group the filtered data by year and name, and calculate the total number of babies
total_babies <- filtered_data %>%
  group_by(year, name) %>%
  summarize(total_babies = sum(n))

# Plot the line graph
ggplot(total_babies, aes(x = year, y = total_babies, color = name, group = name)) +
  geom_line() +
  labs(title = "Comparison of Boys Named Bryan and Brian",
       x = "Year",
       y = "Total Babies")
```

Output:









Result:

Thus a line graph of various criteria from baby names dataset using R language has been created and verified successfully.

Ex No 12: Use the Batting data set from the Lahman package, gives the batting statistics of every player who has played baseball from 1871 through the present day
a. Create a scatterplot of the number of doubles hit in each year of baseball history.
b. Create a scatterplot of the number of doubles hit in each year, in each league. Show only the leagues _NL‘ and _AL‘, and color the NL blue and the AL red
c. Create boxplots for total runs scored per year in the AL and the NL from 1969 to the present
d. Create a histogram of lifetime batting averages (H/AB) for all players who have at least 1000 career AB‘s.

Aim:

To create a scatterplot in R for various criteria using the Batting dataset from the Lahman package.

Algorithm:

1. Install and load packages:
   Check if the babynames package is installed.
   If not, install and load the package.

2. Load required libraries:
   Load the necessary libraries (dplyr and ggplot2).
3. Create a scatterplot of the number of doubles hit in each year of baseball history.
   a. Filter the dataset to include only the relevant variables: yearID and X2B (doubles).
   b. Group the data by yearID and calculate the total number of doubles hit in each year.
   c. Create a scatterplot with yearID on the x-axis and the total number of doubles on the y-axis.
   d. Add labels and titles to the plot for better interpretation.
4. Create a scatterplot of the number of doubles hit in each year, in each league. Show only the leagues _NL‘ and _AL‘, and color the NL blue and the AL red
   a. Load the Lahman package and the Batting dataset.
   b. Filter the dataset to include only the relevant variables: yearID, lgID (league), and X2B (doubles).
   c. Filter the data to keep only the records where the league is either 'NL' or 'AL'.
   d. Group the filtered data by yearID and lgID, and calculate the total number of doubles hit in each year for each league.
   e. Create a scatterplot with yearID on the x-axis, total number of doubles on the y-axis, and color the points based on the league ('NL' as blue and 'AL' as red).
   f. Add labels and titles to the plot for better interpretation.
5. Create boxplots for total runs scored per year in the AL and the NL from 1969 to the present
   a. Load the Lahman package and the Batting dataset.
   b. Filter the dataset to include only the relevant variables: yearID, lgID (league), and R (total runs scored).
   c. Filter the data to keep only the records where the yearID is from 1969 to the present and the league is either 'NL' or 'AL'.
   d. Group the filtered data by yearID and lgID, and calculate the total runs scored in each year for each league.
   e. Create boxplots for the total runs scored per year in the AL and NL separately.
   f. Add labels and titles to the plot for better interpretation.
6. Create a histogram of lifetime batting averages (H/AB) for all players who have at least 1000 career AB‘s.
   a. Load the Lahman package and the Batting dataset.
   b. Filter the dataset to include only the relevant variables: playerID, AB, and H (hits).

c. Group the data by playerID and calculate the total at-bats (AB) and hits (H) for each player.
d. Filter the data to keep only players with at least 1000 career at-bats.
e. Calculate the lifetime batting average (H/AB) for each player.
f. Create a histogram of the lifetime batting averages.

Code:

```
# Load the Lahman package
if (!require("Lahman")) {
  install.packages("Lahman")
  library("Lahman")
}

#a. Create a scatterplot of the number of doubles hit in each year of baseball history.
# Load required libraries
library(dplyr)
library(ggplot2)

# Filter the Batting dataset to include only yearID and X2B (doubles)
batting_doubles <- Batting %>%
  select(yearID, X2B)

# Group the data by yearID and calculate the total number of doubles hit in each year
total_doubles <- batting_doubles %>%
  group_by(yearID) %>%
  summarize(total_doubles = sum(X2B, na.rm = TRUE))

# Create a scatterplot of the number of doubles hit in each year of baseball history
ggplot(total_doubles, aes(x = yearID, y = total_doubles)) +
  geom_point() +
  labs(
    title = "Number of Doubles Hit in Each Year of Baseball History",
    x = "Year",
    y = "Total Doubles"
  )

#b Create a scatterplot of the number of doubles hit in each year, in each league. Show only the leagues _NL‘ and _AL‘, and color the NL blue and the AL red

# Load the Lahman package
if (!require("Lahman")) {
  install.packages("Lahman")
  library("Lahman")
}

# Load required libraries
library(dplyr)
library(ggplot2)

# Filter the Batting dataset to include only yearID, lgID (league), and X2B (doubles)
batting_subset <- Batting %>%
```

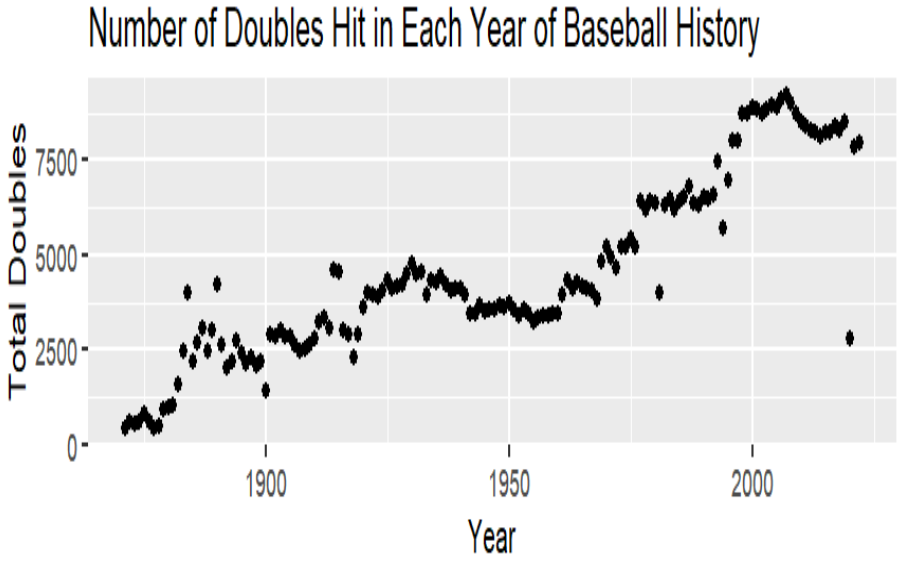
```
  select(yearID, lgID, X2B)

# Filter the data to include only NL and AL leagues
batting_filtered <- batting_subset %>%
  filter(lgID %in% c("NL", "AL"))

# Group the data by yearID and lgID, and calculate the total number of doubles hit in each year for
each league
total_doubles_league <- batting_filtered %>%
  group_by(yearID, lgID) %>%
  summarize(total_doubles = sum(X2B, na.rm = TRUE))

# Create a scatterplot with color differentiation for NL and AL leagues
ggplot(total_doubles_league, aes(x = yearID, y = total_doubles, color = lgID)) +
  geom_point() +
  scale_color_manual(values = c("NL" = "blue", "AL" = "red")) +
  labs(
    title = "Number of Doubles Hit in Each Year by League",
    x = "Year",
    y = "Total Doubles",
    color = "League"
  )

#c Create boxplots for total runs scored per year in the AL and the NL from 1969 to the present

# Load the Lahman package
if (!require("Lahman")) {
  install.packages("Lahman")
  library("Lahman")
}

# Load required libraries
library(dplyr)
library(ggplot2)

# Filter the Batting dataset to include only relevant variables
batting_subset <- Batting %>%
  select(yearID, lgID, R)

# Filter the data to include years from 1969 to present and only NL and AL leagues
filtered_data <- batting_subset %>%
  filter(yearID >= 1969, lgID %in% c("NL", "AL"))

# Group the filtered data by yearID and lgID, and calculate total runs scored in each year for each
league
total_runs_per_year <- filtered_data %>%
  group_by(yearID, lgID) %>%
  summarize(total_runs = sum(R, na.rm = TRUE))

# Create boxplots for total runs scored per year in AL and NL separately
ggplot(total_runs_per_year, aes(x = factor(yearID), y = total_runs, fill = lgID)) +
  geom_boxplot() +
```

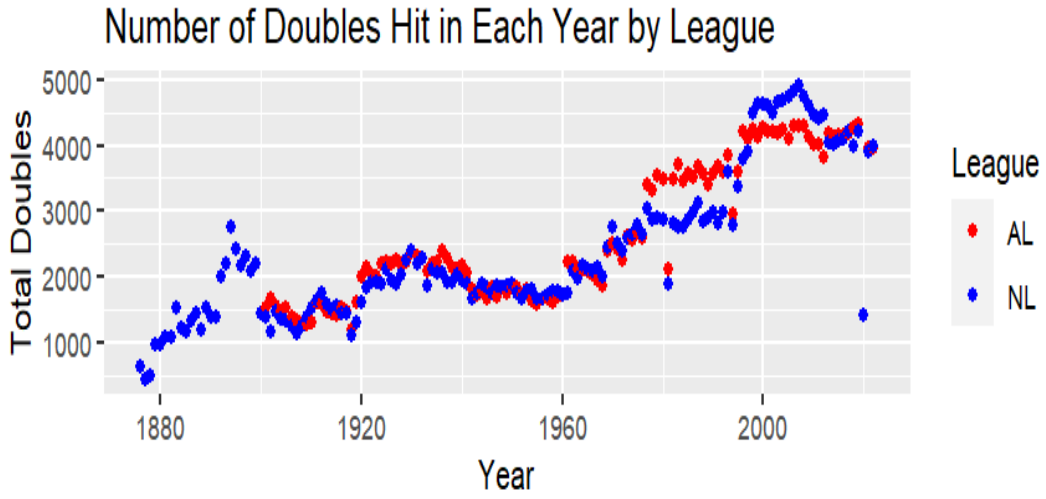
```
  scale_fill_manual(values = c("NL" = "blue", "AL" = "red")) +
  labs(
    title = "Total Runs Scored per Year in AL and NL (1969-present)",
    x = "Year",
    y = "Total Runs",
    fill = "League"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1))
```

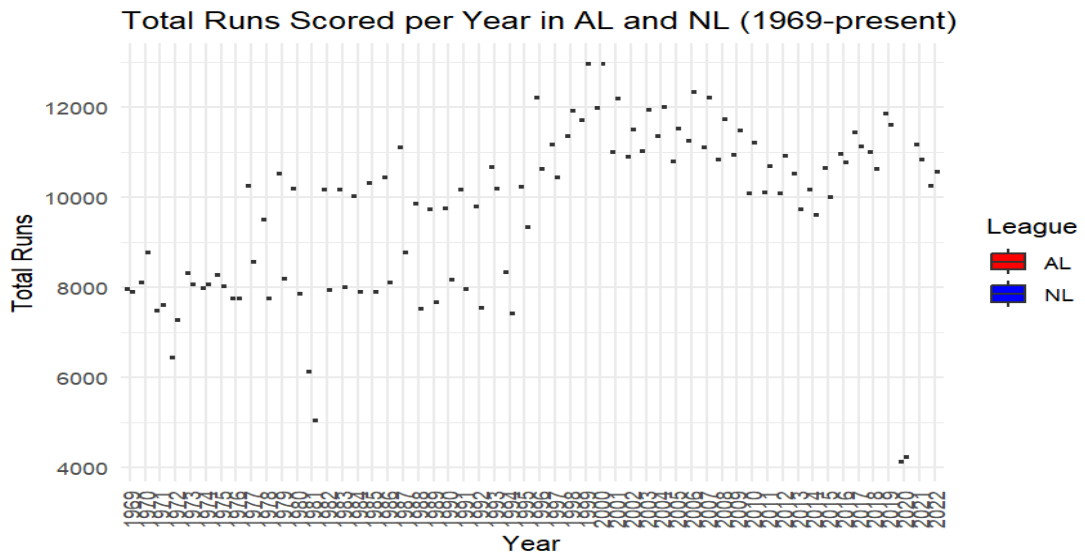
#d Create a histogram of lifetime batting averages (H/AB) for all players who have at least 1000 career AB‘s.

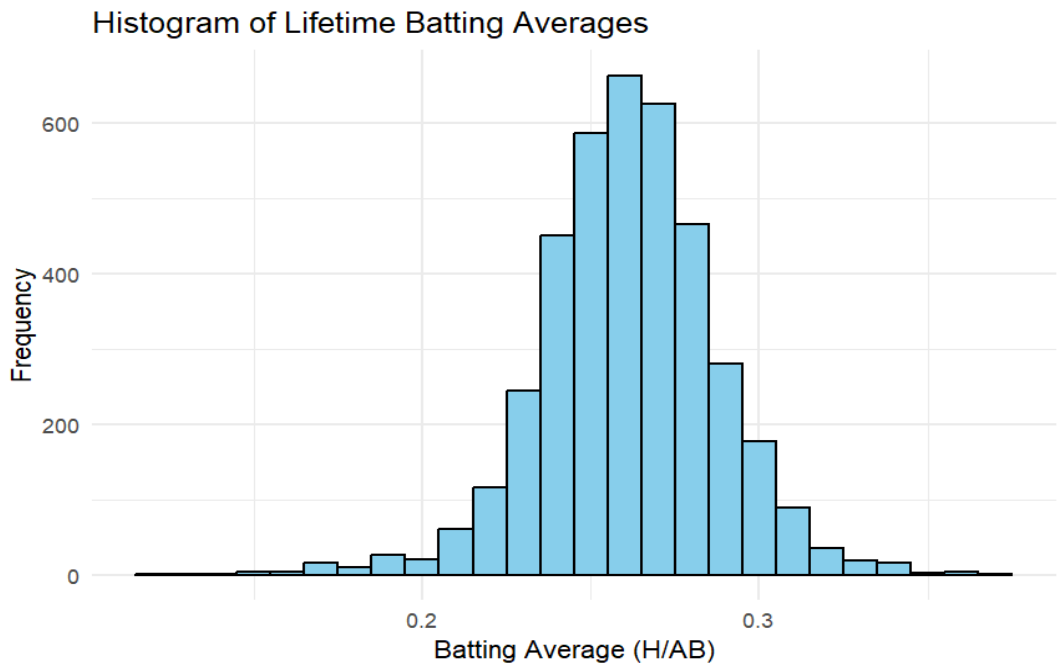
```
# Load the Lahman package
if (!require("Lahman")) {
  install.packages("Lahman")
  library("Lahman")
}

# Load required libraries
library(dplyr)
library(ggplot2)

# Filter the Batting dataset to include relevant variables
batting_subset <- Batting %>%
  select(playerID, AB, H)

# Group the data by playerID and calculate total at-bats (AB) and hits (H) for each player
career_stats <- batting_subset %>%
  group_by(playerID) %>%
  summarize(total_AB = sum(AB, na.rm = TRUE),
            total_H = sum(H, na.rm = TRUE))

# Filter the data to keep only players with at least 1000 career at-bats
career_stats_filtered <- career_stats %>%
  filter(total_AB >= 1000)

# Calculate lifetime batting average (H/AB) for each player
career_stats_filtered <- career_stats_filtered %>%
  mutate(batting_avg = total_H / total_AB)

# Create a histogram of lifetime batting averages
ggplot(career_stats_filtered, aes(x = batting_avg)) +
  geom_histogram(binwidth = 0.01, fill = "skyblue", color = "black") +
  labs(
    title = "Histogram of Lifetime Batting Averages",
    x = "Batting Average (H/AB)",
    y = "Frequency"
  ) +
  theme_minimal()
```

Output:


Number of Doubles Hit in Each Year of Baseball History
Total Doubles
7500
5000
2500
0
1900
1950
2000
Year


Number of Doubles Hit in Each Year by League
Total Doubles
5000
4000
3000
2000
1000
1880
1920
1960
2000
Year
League
AL
NL


Total Runs Scored per Year in AL and NL (1969-present)
Total Runs
12000
10000
8000
6000
4000
Year
League
AL
NL



Result:

Thus a scatterplot in R for various criteria using the Batting dataset from the Lahman package has been created and executed successfully.